1. A list is an ordered and mutable collection in Python.
2. Lists are used to store multiple values in one variable.
3. Lists are created using square brackets `[]`.
4. Lists allow duplicate elements.
5. Lists support different data types together.
6. Positive indexing starts from `0`.
7. Negative indexing starts from `-1`.
8. Lists support slicing using `[start:stop:step]`.
9. Python lists internally work like dynamic arrays.
10. Common list methods are `append()`, `extend()`, `remove()`, and `pop()`.

In [13]:
```
help(list)
```

```
Help on class list in module builtins:

class list(object)
 |  list(iterable=(), /)
 |
 |  Built-in mutable sequence.
 |
 |  If no argument is given, the constructor creates a new empty list.
 |  The argument must be an iterable if specified.
 |
 |  Methods defined here:
 |
 |  __add__(self, value, /)
 |      Return self+value.
 |
 |  __contains__(self, key, /)
 |      Return bool(key in self).
 |
 |  __delitem__(self, key, /)
 |      Delete self[key].
 |
 |  __eq__(self, value, /)
 |      Return self==value.
 |
 |  __ge__(self, value, /)
 |      Return self>=value.
 |
 |  __getattribute__(self, name, /)
 |      Return getattr(self, name).
 |
 |  __getitem__(self, index, /)
 |      Return self[index].
 |
 |  __gt__(self, value, /)
 |      Return self>value.
 |
 |  __iadd__(self, value, /)
 |      Implement self+=value.
 |
 |  __imul__(self, value, /)
 |      Implement self*=value.
 |
 |  __init__(self, /, *args, **kwargs)
 |      Initialize self.  See help(type(self)) for accurate signature.
 |
 |  __iter__(self, /)
 |      Implement iter(self).
 |
 |  __le__(self, value, /)
 |      Return self<=value.
 |
 |  __len__(self, /)
 |      Return len(self).
 |
 |  __lt__(self, value, /)
 |      Return self<value.
 |
 |  __mul__(self, value, /)
 |      Return self*value.
 |
 |  __ne__(self, value, /)
 |      Return self!=value.
 |
 |  __repr__(self, /)
 |      Return repr(self).
 |
 |  __reversed__(self, /)
 |      Return a reverse iterator over the list.
 |
 |  __rmul__(self, value, /)
 |      Return value*self.
 |
 |  __setitem__(self, key, value, /)
 |      Set self[key] to value.
 |
 |  __sizeof__(self, /)
 |      Return the size of the list in memory, in bytes.
 |
 |  append(self, object, /)
 |      Append object to the end of the list.
 |
 |  clear(self, /)
 |      Remove all items from list.
 |
 |  copy(self, /)
 |      Return a shallow copy of the list.
 |
 |  count(self, value, /)
 |      Return number of occurrences of value.
 |
```

```
|  extend(self, iterable, /)
|      Extend list by appending elements from the iterable.
|
|  index(self, value, start=0, stop=9223372036854775807, /)
|      Return first index of value.
|
|      Raises ValueError if the value is not present.
|
|  insert(self, index, object, /)
|      Insert object before index.
|
|  pop(self, index=-1, /)
|      Remove and return item at index (default last).
|
|      Raises IndexError if list is empty or index is out of range.
|
|  remove(self, value, /)
|      Remove first occurrence of value.
|
|      Raises ValueError if the value is not present.
|
|  reverse(self, /)
|      Reverse *IN PLACE*.
|
|  sort(self, /, *, key=None, reverse=False)
|      Sort the list in ascending order and return None.
|
|      The sort is in-place (i.e. the list itself is modified) and stable (i.e. the
|      order of two equal elements is maintained).
|
|      If a key function is given, apply it once to each list item and sort them,
|      ascending or descending, according to their function values.
|
|      The reverse flag can be set to sort in descending order.
|
|  ----------------------------------------------------------------------
|  Class methods defined here:
|
|  __class_getitem__(...)
|      See PEP 585
|
|  ----------------------------------------------------------------------
|  Static methods defined here:
|
|  __new__(*args, **kwargs)
|      Create and return a new object.  See help(type) for accurate signature.
|
|  ----------------------------------------------------------------------
|  Data and other attributes defined here:
|
|  __hash__ = None
```

List Creation

direct method

In [115…
```
list = [] # Empty List
list
```

Out[115…
```
[]
```

indirect method

In [118…
```
a = list((1,2,3))

print(a)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[118], line 1
----> 1 a = list((1,2,3))
      3 print(a)

TypeError: 'list' object is not callable
```

In [120…
```
del list
```

In [122…
```
a = list((1,2,3))

print(a)
```

```
[1, 2, 3]
```

2 Convert String to List

In [125…
```
a = list("Python")

print(a)
```
```
['P', 'y', 't', 'h', 'o', 'n']
```

3. Convert Tuple to List

In [128…
```
t = (10,20,30)

a = list(t)

print(a)
```
```
[10, 20, 30]
```

4. Convert Set to List

In [131…
```
s = {1,2,3}

a = list(s)

print(a)
```
```
[1, 2, 3]
```

In [47]:
```
# Creation of a List in Python

# Creating an empty List
empty_List = []

# Creating a List of Integers
integer_List = [26, 12, 97, 8]

# Creating a List of floating point numbers
float_List = [5.8, 12.0, 9.7, 10.8]

# Creating a List of Strings
string_List = ["Interviewbit", "Preparation", "India"]

# Creating a List containing items of different data types
List = [1, "Interviewbit", 9.5, 'D']

# Creating a List containing duplicate items
duplicate_List = [1, 2, 1, 1, 3,3, 5, 8, 8]
```

List Operator in Python

In [50]:
```
List1 = [1, 2, 3, 4]
List2 = [5, 6, 7, 8]
concat_List = List1 + List2
print(concat_List)
```
```
[1, 2, 3, 4, 5, 6, 7, 8]
```

In [52]:
```
my_List = [1, 2, 3, 4]
print(my_List*2)
```
```
[1, 2, 3, 4, 1, 2, 3, 4]
```

In [ ]:
```
```

List Membership

Membership means checking whether an element exists in a list.

In [42]:
```
list1=['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']
list1
```
Out[42]:
```
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']
```

In [44]:
```
'one' in list1 # Check if 'one' exist in the list
```
Out[44]:
```
True
```

In [46]:
```
'ten' in list1 # Check if 'ten' exist in the list
```
Out[46]:
```
False
```

In [54]:
```
if 'three' in list1:# Check if 'three' exist in the list
 print('Three is present in the list')
```

```
else:
    print('Three is not present in the list')
```

```
Three is present in the list
```

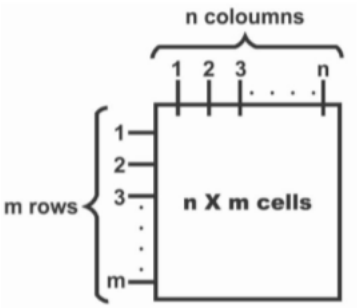
In [60]:
```
if 'eleven' in list1: # Check if 'eleven' exist in the list
 print('eleven is present in the list')
else:
   print('eleven is not present in the list')
```

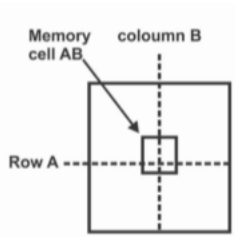
```
eleven is not present in the list
```

In [55]:
```
my_List = [1, 2, 3, 4, 5, 6]
print(3 in my_List)
print(10 in my_List)
print(10 not in my_List)
```

```
True
False
True
```

Memory address





- Memory address is the location of an object in RAM
- `id()` function gives object identity
- Same object → same memory address
- Different objects → different memory addresses
- Mutable objects can share same reference

id() Function

The id() function returns the memory address (identity) of an object.

In [48]:
```
a = [10,20,30]
a
```

Out[48]:
```
[10, 20, 30]
```

In [50]:
```
print(id(a))
```

```
2117512521152
```

Here:

- `a` is a variable
- List object is created in memory
- `id(a)` gives address of that object

```
a  -----> [1,2,3]
          ↑
       memory address
```

`id(a)` returns this address.

In [53]:
```
a = [10,20,30]
print(id(a))
```

```
2117512663168
```

In [55]:
```
a = [1,2,3]

b = a
print(a)
print(b)
print(id(a))
print(id(b))
```

```
[1, 2, 3]
[1, 2, 3]
2117512607552
2117512607552
```

memory address is same

Both IDs same because:

- `a` and `b` point to same list object.

In [58]:
```
a = [1,2,3]

b = a.copy()

print(id(a))
print(id(b))
```

```
2117512584960
2117512590080
```

Different IDs because copy creates new object.

In [61]:
```
a = [1,2]
b = [1,2]

print(a)
print(b)
print(a == b)# Are the VALUES same?"
```

```
[1, 2]
[1, 2]
True
```

In [63]:
```
print(id(a))
print(id(b))
print(a is b)  #Are both variables pointing to Same memory object?"
```

```
2117512592512
2117512605120
False
```

- `True` → same object
- `False` → different object

Difference Between `==` and `is`

Operator	Meaning
`==`	Checks values
`is`	Checks memory/object

In [66]:
```
L = [1,2,3]

print(id(L))
print(id(L[0]))# index location
print(id(L[1]))
print(id(L[2]))
```

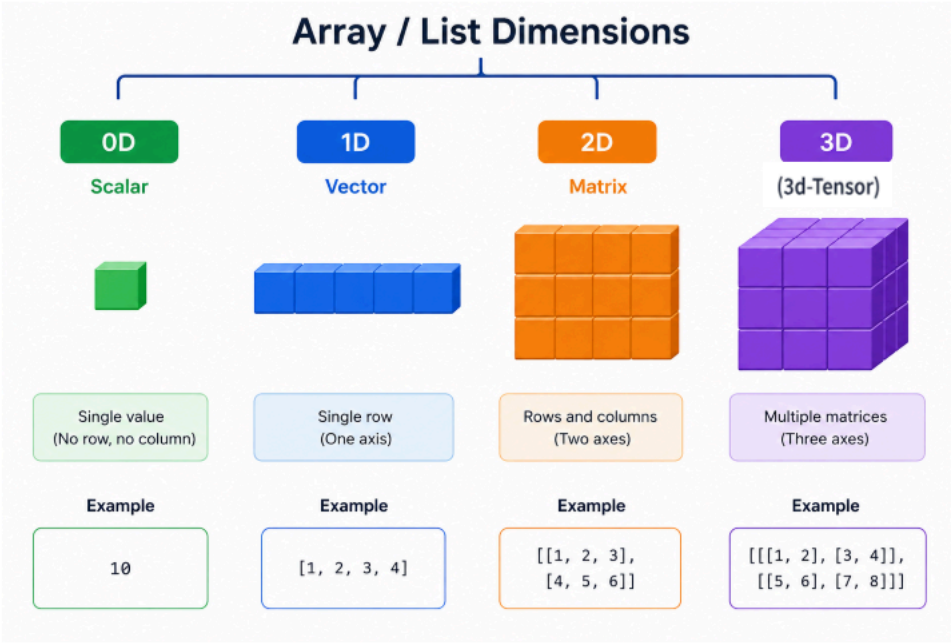
```
2117512494400
140718878046648
140718878046680
140718878046712
```

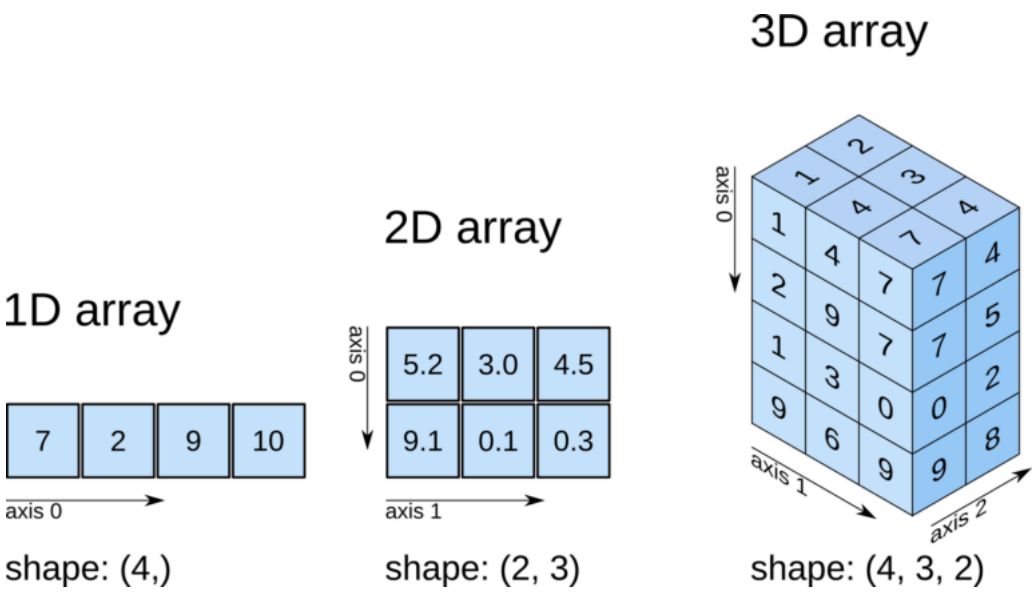
In [68]:
```
print(id(1))# value
print(id(2))
print(id(3))
```

```
140718878046648
140718878046680
140718878046712
```

---

dimensions





In [71]:
```
#0D (Scalar)
a = 10

print(a)
```

```
10
```

In [73]:
```
# Scalar (0D)
scalar = 10

# Vector (1D)
vector = [1, 2, 3]

# Matrix (2D)
matrix = [
    [1, 2],
    [3, 4]
]

# Tensor (3D)
tensor = [
    [
        [1, 2],
        [3, 4]
    ]
]

print("Scalar (0D):", scalar)
print()

print("Vector (1D):", vector)
```

```
print()

print("Matrix (2D):")
print(matrix)
print()

print("Tensor (3D):")
print(tensor)
```

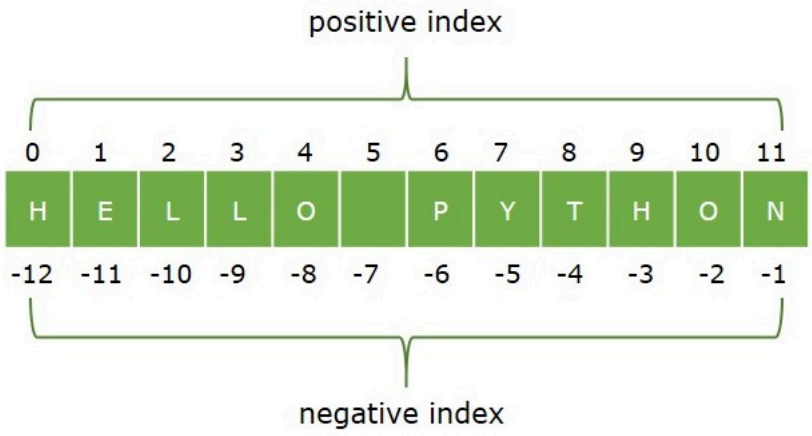
```
Scalar (0D): 10

Vector (1D): [1, 2, 3]

Matrix (2D):
[[1, 2], [3, 4]]

Tensor (3D):
[[[1, 2], [3, 4]]]
```

Accessing Element

Accessing elements means retrieving or reading values stored inside a list using indexes.

Python lists store data in sequence, and each element has a position called an index.

Indexing



1. Positive Indexing

Positive indexing starts from left to right.

```
Index:   0    1    2    3
List : [10, 20, 30, 40]
```

- First element → index `0`
- Second element → index `1`

2. Negative Indexing

Negative indexing starts from right to left.

```
Index:   -4   -3   -2   -1
List : [10, 20, 30, 40]
```

- Last element → `-1`
- Second last → `-2`

In [97]:
```
l = [10,20,44,55,22]
#    0  1  2  3  4
print(l)
```

```
[10, 20, 44, 55, 22]
```

In [99]:
```
print(l[0])
print(type(l[0]))
```

```
10
<class 'int'>
```

In [101…
```
print(l[4])
```

```
22
```

In [103…
```
print(l[5])
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[103], line 1
----> 1 print(l[5])

IndexError: list index out of range
```

In [105…
```
print(l[10])
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[105], line 1
----> 1 print(l[10])

IndexError: list index out of range
```

In [107…
```
n = 14

l = [1,2,3,4,5,n]

print(l)
```

```
[1, 2, 3, 4, 5, 14]
```

In [109…
```
l = [1,2,3,4,5]

l[2] = 55

print(l)
```

```
[1, 2, 55, 4, 5]
```

In [113…
```
l[5] = 100
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[113], line 1
----> 1 l[5] = 100

IndexError: list assignment index out of range
```

In [111…
```
#    -4    -3      -2     -1
l = [12, 'indore', True, 'Goa']
#     0      1       2     3
print(type(l[2]))
```

```
<class 'bool'>
```

In [117…
```
print(type(l[1]))
print(l[-1])
print(l[-4])
```

```
<class 'str'>
Goa
12
```

In [144…
```
# Indexing
L = [[[1,2],[3,4]],[[5,6],[7,8]]]
#positive
print(L[0][0][1])

# Slicing
L = [1,2,3,4,5,6]

print(L[::-1])
```

```
2
[6, 5, 4, 3, 2, 1]
```

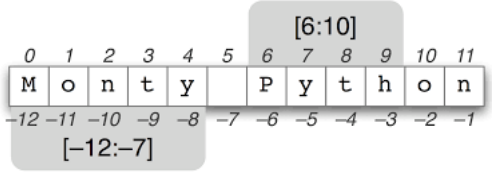
In [146…
```
# indexing
L=[[[1,2],[3,4]],[[5,6],[7,8]]]
#positive
print(L[0][0][1])

# slicing
L=[1,2,3,4,5,6]

print(L[::-1])


#indexing
L=[[[1,2],[3,4]],[[5,6],[7,8]]]

#positive
print(L[0][0][1])

# slicing
L=[1,2,3,4,5,6]
print(L[::-1])
```

```
2
[6, 5, 4, 3, 2, 1]
2
[6, 5, 4, 3, 2, 1]
```

In [ ]:

In [ ]:

Slicing a Python List



In [119…
```
l = [1,4,9,16,32,49,64,100,120,300]
#    0 1 2  3  4 5  6   7   8   9
print(l[2:6])   # start : end
```

```
[9, 16, 32, 49]
```

In [122…
```
print(l)
```

```
[1, 4, 9, 16, 32, 49, 64, 100, 120, 300]
```

In [124…
```
print(l[1:3])
```

```
[4, 9]
```

In [126…
```
print(l[:8])
```
```
[1, 4, 9, 16, 32, 49, 64, 100]
```

In [128…
```
print(l[5:])
```
```
[49, 64, 100, 120, 300]
```

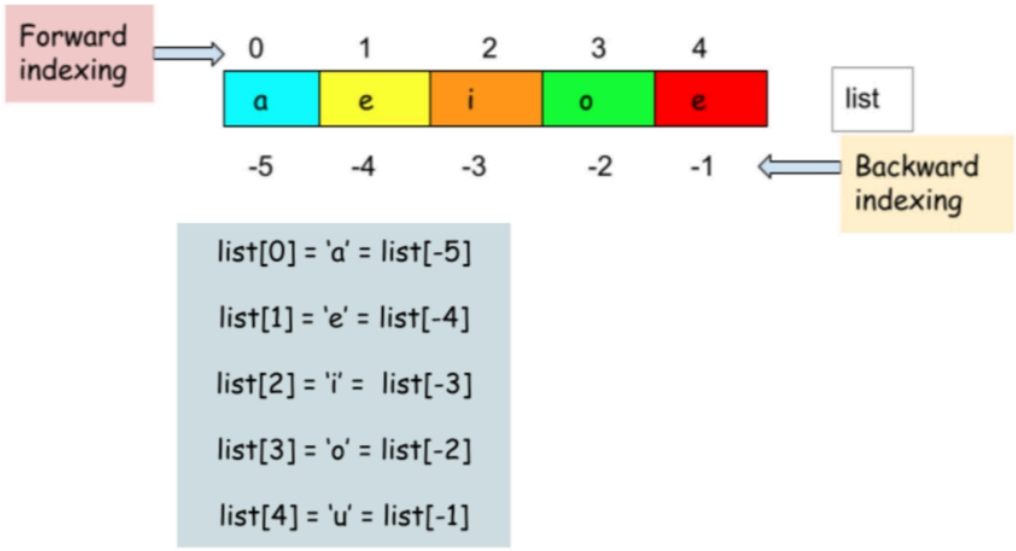
In [130…
```
print(l[:])
print(l[0:0])
print(l[4:1])
```
```
[1, 4, 9, 16, 32, 49, 64, 100, 120, 300]
[]
[]
```

In [132…
```
print(l[-4:-1])
```
```
[64, 100, 120]
```

In [134…
```
print(l[-1:-5])
```
```
[]
```

Important Rules — Positive Index & Negative Index

```
Positive :   0    1    2    3    4   Starts from Left → Right
Values   : [10, 20, 30, 40, 50]
Negative :  -5   -4   -3   -2   -1   Starts from Right → Left
```



list[0] = 'a' = list[-5]

list[1] = 'e' = list[-4]

list[2] = 'i' = list[-3]

list[3] = 'o' = list[-2]

list[4] = 'u' = list[-1]

Python

```python
a = [10, 20, 30, 40, 50, 60]
```

Example	Output	Explanation
`a[0]`	`10`	First element
`a[2]`	`30`	Element at index 2
`a[-1]`	`60`	Last element
`a[-2]`	`50`	Second last element
`a[1:4]`	`[20,30,40]`	Index 1 to 3
`a[:3]`	`[10,20,30]`	Start to index 2
`a[3:]`	`[40,50,60]`	Index 3 to end
`a[::2]`	`[10,30,50]`	Every 2nd element
`a[1::2]`	`[20,40,60]`	Alternate elements
`a[::-1]`	`[60,50,40,30,20,10]`	Reverse list
`a[::-2]`	`[60,40,20]`	Reverse alternate
`a[4:1:-1]`	`[50,40,30]`	Backward traversal
`a[5:0:-2]`	`[60,40,20]`	Reverse with step 2
`a[0:4:1]`	`[10,20,30,40]`	Forward traversal
`a[4:0:1]`	`[]`	Invalid positive traversal
`a[0:4:-1]`	`[]`	Invalid negative traversal
`a[-4:-1]`	`[30,40,50]`	Negative slicing

`a[5:0:-2]`	`[60,40,20]`	Reverse with step 2
`a[0:4:1]`	`[10,20,30,40]`	Forward traversal
`a[4:0:1]`	`[]`	Invalid positive traversal
`a[0:4:-1]`	`[]`	Invalid negative traversal
`a[-4:-1]`	`[30,40,50]`	Negative slicing
`a[-1:-5:-1]`	`[60,50,40,30]`	Reverse negative slicing
`a[2:3]`	`[30]`	Slice returns list
`a[2]`	`30`	Index returns element

Rule	Meaning
Positive index	Left → Right
Negative index	Right → Left
Stop index	Always excluded
Positive step	`start < stop`
Negative step	`start > stop`
`[::-1]`	Reverse list
`[:]`	Full shallow copy

$$S[start:stop:step]$$

Start position *End position* *The increment*

In [136…
```
print(l[0:9:2])   # start : end : steps
```
```
[1, 9, 32, 64, 120]
```

In [138…
```
print(l[-4:-1:-1])
```
```
[]
```

In [140…
```
print(l[-4:-1:1])
```
```
[64, 100, 120]
```

negative steps

In [163…
```
a = [1,2,3,4,5]
print(a[::-1]) #full reverse
```
```
[5, 4, 3, 2, 1]
```

In [165…
```
a = [1,2,3,4,5]
print(a[::1])
```
```
[1, 2, 3, 4, 5]
```

In [ ]:

In [ ]:

Most Important Interview Points

- Stop index is always excluded.
- Negative index starts from end.
- `[:]` creates shallow copy.
- Slicing returns new list.
- Index access → O(1)
- Slicing → O(n)
- `[::-1]` reverses list.

List Functions

A **List Function** is a built-in Python function that performs operations on lists.

These functions:

- Work with list objects
- Are called directly
- Do NOT use dot (`.`) notation

Function	Purpose
len()	Count elements
type()	Check datatype
max()	Largest element
min()	Smallest element
sum()	Total sum
sorted()	Return sorted list
list()	Create/convert list
any()	At least one True
all()	All values True
enumerate()	Index with values
zip()	Combine iterables
reversed()	Reverse iterator
id()	Memory address

list()

Convert other datatypes into a list

"Convert iterable data into list format."

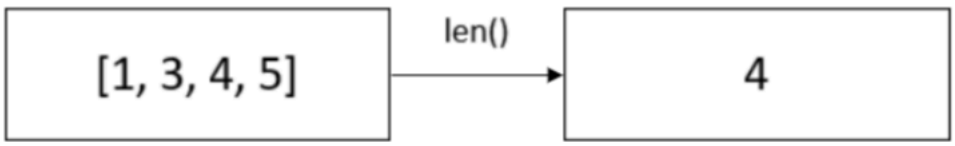
note:-coverd in list creation

id()

covred in id function

1. len()

We can find the total number of elements in a list using the length function.



In [6]:
```
l=[]
```

In [8]:
```
len(l)
```

Out[8]: 0

In [10]:
```
kunal=["india", 1, 2,12.2]
```

In [12]:
```
len(kunal)
```

Out[12]: 4

In [14]:
```
list=[1,34,56.34,"prakash",{1,2}]
```

In [16]:
```
len(list)
```

Out[16]: 5

In [18]:
```
a = [10,20,30,40]

print (len(a))
```

```
4
```

2 type

type() is used to check the datatype.

In [21]:
```
a = [10, 20, 30]

print(type(a))
```

```
<class 'list'>
```

In [23]:
```
students = ["KUNAL", "RAHUL", "AMIT"]

print(type(students))
```

```
<class 'list'>
```

In [25]:
```
# INTEGER
a = 10
print(type(a))

# FLOAT
b = 10.5
print(type(b))

# STRING
c = "KUNAL"
print(type(c))

# LIST
d = [1, 2, 3]
print(type(d))

# TUPLE
e = (1, 2, 3)
print(type(e))

# SET
f = {1, 2, 3}
print(type(f))

# DICTIONARY
g = {"name": "KUNAL", "age": 25}
print(type(g))

# BOOLEAN
h = True
print(type(h))

# COMPLEX
i = 5 + 6j
print(type(i))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'list'>
<class 'tuple'>
<class 'set'>
<class 'dict'>
<class 'bool'>
<class 'complex'>
```

In [27]:
```
# INTEGER
a = 10
print(a.dtype)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[27], line 3
      1 # INTEGER
      2 a = 10
----> 3 print(a.dtype)

AttributeError: 'int' object has no attribute 'dtype'
```

---

type() vs dtype in Python

dtype

Checks datatype inside NumPy array.

In [ ]:
```python
import numpy as np

a = np.array([1, 2, 3])

print(a.dtype)
```

In [ ]:
```python
import numpy as np

a = np.array([1, 2, 3])
b = [10, 20, 30]

print(type(a))
print(type(b))
print(a.dtype)
```

In [ ]:
```python
#Heterogeneous
a = np.array([1, 2.5, "KUNAL", True])
print(a.dtype)
```

Heterogeneous means:

> Different types of data stored together.

Example:

Python
```python
data = [10, 5.5, "KUNAL", True]
```

Here list contains:

- `10` → int
- `5.5` → float
- `"KUNAL"` → string
- `True` → bool

So this is a **heterogeneous list**.

`type()` vs `dtype` in Python

Feature	`type()`	`dtype`
Used for	Normal Python datatype	NumPy/Pandas datatype
Works on	Any Python object	Arrays, Series, DataFrame
Output	Object class	Data type of elements
Library	Built-in Python	NumPy / pandas

3.max() & min()

max() is used to find the largest value.

min() is used to find the smallest value.

In [36]:
```
a = [10, 50, 20, 90, 30]

print(max(a))
print(min(a))
```

```
90
10
```

In [38]:
```
names = ["KUNAL", "RAHUL", "AMIT"]

print(max(names))#(Compared alphabetically)
print(min(names))#(Compared alphabetically)
```

```
RAHUL
AMIT
```

In [40]:
```
data = [10, 5.5, "KUNAL", True]

print(max(data))
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[40], line 3
      1 data = [10, 5.5, "KUNAL", True]
----> 3 print(max(data))

TypeError: '>' not supported between instances of 'str' and 'int'
```

---

sum()

sum() is used to calculate the total/addition of values.

In [44]:
```
a = [10, 20, 30]

print(sum(a))
```

```
60
```

In [46]:
```
data = [10, 5.5, "KUNAL", True]

print(sum(data))
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[46], line 3
      1 data = [10, 5.5, "KUNAL", True]
----> 3 print(sum(data))

TypeError: unsupported operand type(s) for +: 'float' and 'str'
```

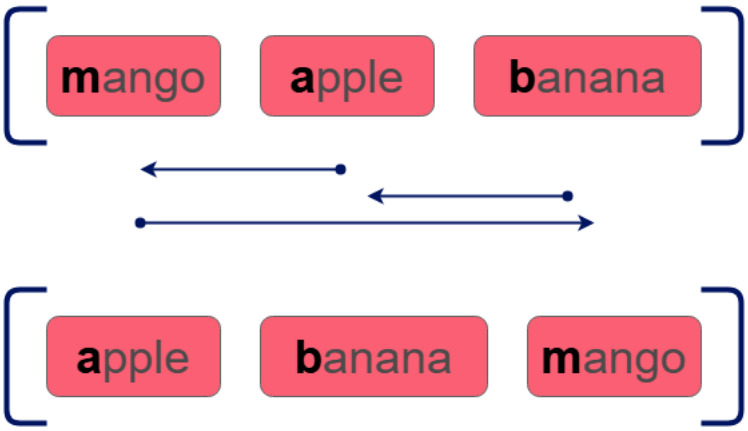
In [48]:
```
data = [10, 5.5, True]

print(sum(data))#True=1
```

```
16.5
```

---

sort() in List

- `sort()` is a list method
- Sorts the original list directly (in-place)
- Default sorting → ascending order
- Returns `None`
- Works only on lists
- Supports `reverse=True` for descending order
- Supports `key=` for custom sorting
- Modifies original list permanently
- Heterogeneous data may give `TypeError`



sort() is a list method used to arrange elements in order.and changes original list permanently.

Ascending → small to large

Descending → large to small

In [52]:
```
a = [50, 10, 40, 20]

a.sort()

print(a) #Ascending → small to large
```

```
[10, 20, 40, 50]
```

In [54]:
```
a = [50, 10, 40, 20]

a.sort(reverse=True) #Descending → large to small

print(a)
```

```
[50, 40, 20, 10]
```

In [56]:
```
names = ["KUNAL", "RAHUL", "AMIT"]

names.sort()

print(names)
```

```
['AMIT', 'KUNAL', 'RAHUL']
```

In [58]:
```
data = [10, "KUNAL", 5.5]

data.sort()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[58], line 3
      1 data = [10, "KUNAL", 5.5]
----> 3 data.sort()

TypeError: '<' not supported between instances of 'str' and 'int'
```

In [60]:
```
a = [3, 1, 2]

print(a.sort())# beacuse sort return nothing
```

```
None
```

sorted()

sorted() is used to sort data and returns a new sorted list.

- `sorted()` is a built-in function
- Used to sort iterable data
- Returns a new sorted list
- Original data remains unchanged
- Works with lists, tuples, strings, sets, dictionaries
- Default sorting → ascending order
- Use `reverse=True` for descending order
- Can sort strings alphabetically
- Returns list type always

Python

```
sorted(iterable, reverse=False)
```

In [63]:
```
a = [50, 10, 40, 20]

a.sorted()

print(a)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[63], line 3
      1 a = [50, 10, 40, 20]
----> 3 a.sorted()
      5 print(a)

AttributeError: 'list' object has no attribute 'sorted'
```

In [65]:
```
a = [50, 10, 40, 20]

b = sorted(a)

print(b)
```

```
[10, 20, 40, 50]
```

In [67]:
```
a = [50, 10, 40, 20]

b = sorted(a)

print(a)   #Original List Remains Same
print(b)
```

```
[50, 10, 40, 20]
[10, 20, 40, 50]
```

In [69]:
```
a = [50, 10, 40, 20]

print(sorted(a, reverse=True)) #Descending Order
```

```
[50, 40, 20, 10]
```

In [71]:
```
names = ["KUNAL", "RAHUL", "AMIT"]

print(sorted(names))
```

```
['AMIT', 'KUNAL', 'RAHUL']
```

In [73]:
```
# sorted by text
text = "KUNAL"

print(sorted(text))
```

```
['A', 'K', 'L', 'N', 'U']
```

In [75]:
```
# sort()
a = [50, 10, 40]

a.sort()

print(a)# original list change

#sorted()
a = [50, 10, 40]

b = sorted(a)

print(a) # original list unchanged
print(b)
```

```
[10, 40, 50]
[50, 10, 40]
[10, 40, 50]
```

In [77]:
```
data = [10, "KUNAL", 5.5, True]

print(sorted(data))
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[77], line 3
      1 data = [10, "KUNAL", 5.5, True]
----> 3 print(sorted(data))

TypeError: '<' not supported between instances of 'str' and 'int'
```

a.sort()== modifies same list

sorted(a)==creates new list

Main Difference: `sort()` vs `sorted()`

Feature	`sort()`	`sorted()`
Type	List method	Built-in function
Original data changes?	Yes	No
Return value	`None`	New sorted list
Works on	Only list	List, tuple, set, string

==========

any()

> Python `any()` function returns `True` if at least one element in an iterable (List, Tuple, Set, Dictionary, etc.) is `True`. Otherwise, it returns `False`.

Syntax: *any(iterable)*

- ***Iterable:*** *It is an iterable object such as a dictionary, tuple, list, set, etc.*

Returns: *Returns True if any of the items is True.*

In [88]:
```
# a List of boolean values
l = [False, False, True, False, False]
```

```
print(any(l))
```

```
True
```

In [92]:
```
# Some elements of list are True while others are False
 l = [1, 0, 6, 7, False]
 print(any(l))
```

```
  Cell In[92], line 2
    l = [1, 0, 6, 7, False]
    ^
IndentationError: unexpected indent
```

In [94]:
```
# All elements of list are False
l = [0, 0, False]
print(any(l))
```

```
False
```

In [96]:
```
# All elements of list are True
l = [4, 5, 1]
print(any(l))
```

```
True
```

In [90]:
```
# Empty list
l = []
print(any(l))
```

```
False
```

In [98]:
```
#In dictionaries, any() checks only the keys.
d = {0: "A", 1: "B"}

print(any(d))
```

```
True
```

all()

`all()` is a Python built-in function.

It returns `True` only if **all elements** in an iterable are `True`.

If even one element is `False`, it returns `False`.

In [103…
```
a = [1, 2, 3]

print(all(a)) # Because all non-zero numbers are True.
```

```
True
```

In [105…
```
a = [1, 0, 3]

print(all(a)) # beacause 0 is false
```

```
False
```

In [111…
```
a = ["Python", "Java", ""]

print(all(a)) # empty string is false
```

```
False
```

In [113…
```
a = ["Python", "Java", "php"]

print(all(a)) # empty string
```

```
True
```

In [115…
```
## dict
d = {1: "A", 2: "B"}

print(all(d)) # all() checks only dictionary keys.
```

```
True
```

Difference Between `all()` and `any()` in Python

Feature	`any()`	`all()`
Meaning	At least one True	All must be True
Returns `True` when	One element is True	Every element is True
Returns `False` when	All are False	Even one is False
Logic	OR logic	AND logic

enumrate()

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

zip()

zip() is an inbuilt Python function used to combine multiple iterables (like lists, tuples, sets, strings) element by element.

It creates pairs/groups based on the same index position.

```
zip(list1, list2)
```

It pairs items based on their index.

In [7]:
```
#Two Lists
a = [1, 2, 3]
b = ['A', 'B', 'C']

z = zip(a, b)

print(list(z))
```

```
[(1, 'A'), (2, 'B'), (3, 'C')]
```

In [9]:
```
## More Than Two Lists
name = ['Kunal', 'Rahul', 'Aman']
age = [25, 30, 22]
city = ['Pune', 'Mumbai', 'Delhi']

print(list(zip(name, age, city)))
```

```
[('Kunal', 25, 'Pune'), ('Rahul', 30, 'Mumbai'), ('Aman', 22, 'Delhi')]
```

Memory Efficient

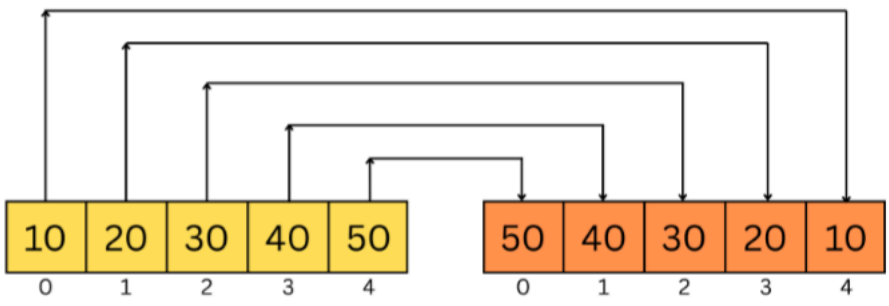
`zip()` is memory efficient because:

✅ Returns iterator

✅ Generates values one-by-one

reverse()

reverse() is a list method used to reverse the order of elements in the same list.



- `reverse()` is a list method
- Reverses the original list
- Modifies list in-place
- Does not create new list
- Returns `None`
- Works only with lists
- Changes element order from end to start

```
list.reverse()
```

In [17]:
```
a = [1, 2, 3, 4]

a.reverse()

print(a)
```

```
[4, 3, 2, 1]
```

In [19]:
```
name = ['Kunal', 'Rahul', 'Aman']

name.reverse()

print(name)
```

```
['Aman', 'Rahul', 'Kunal']
```

In [79]:
```
# Reversing a list of numbers
numbers = [1, 2, 3, 4, 5]
reversed_numbers = numbers[::-1]
print(reversed_numbers)

# Reversing a list of characters
characters = ['a', 'b', 'c', 'd']
reversed_characters = characters[::-1]
print(reversed_characters)
```

```
[5, 4, 3, 2, 1]
['d', 'c', 'b', 'a']
```

In [ ]:

reversed()

`reversed()` is an inbuilt Python function used to reverse an iterable.

Unlike `reverse()`:

✅ It does NOT modify original data
✅ It returns an iterator

- `reversed()` is a built-in function
- Returns a reverse iterator
- Does not modify original data
- Works with lists, tuples, strings
- To see output properly, convert to `list()`, `tuple()`, or `str`
- Opposite of normal iteration

Syntax

Python

```
reversed(iterable)
```

In [22]:
```
a = [1,2,3]

r = reversed(a)

print(list(r))
```

```
[3, 2, 1]
```

In [26]:
```
a
```

Out[26]:
```
[1, 2, 3]
```

Difference: `reverse()` vs `reversed()`

reverse()	reversed()
List method	Built-in function
Changes original list	Does not change original
Returns None	Returns iterator
Only for list	Works with many iterables

==========

List methods

A **List Method** is a built-in function specially designed to perform operations on list objects.

These methods are attached to the list object and are called using dot (`.`) notation.

Method	Purpose
append()	Add single element
extend()	Add multiple elements
insert()	Insert at specific position
remove()	Remove value
pop()	Remove using index
clear()	Remove all elements
index()	Find position
count()	Count occurrences
sort()	Sort list
reverse()	Reverse list
copy()	Copy list

---

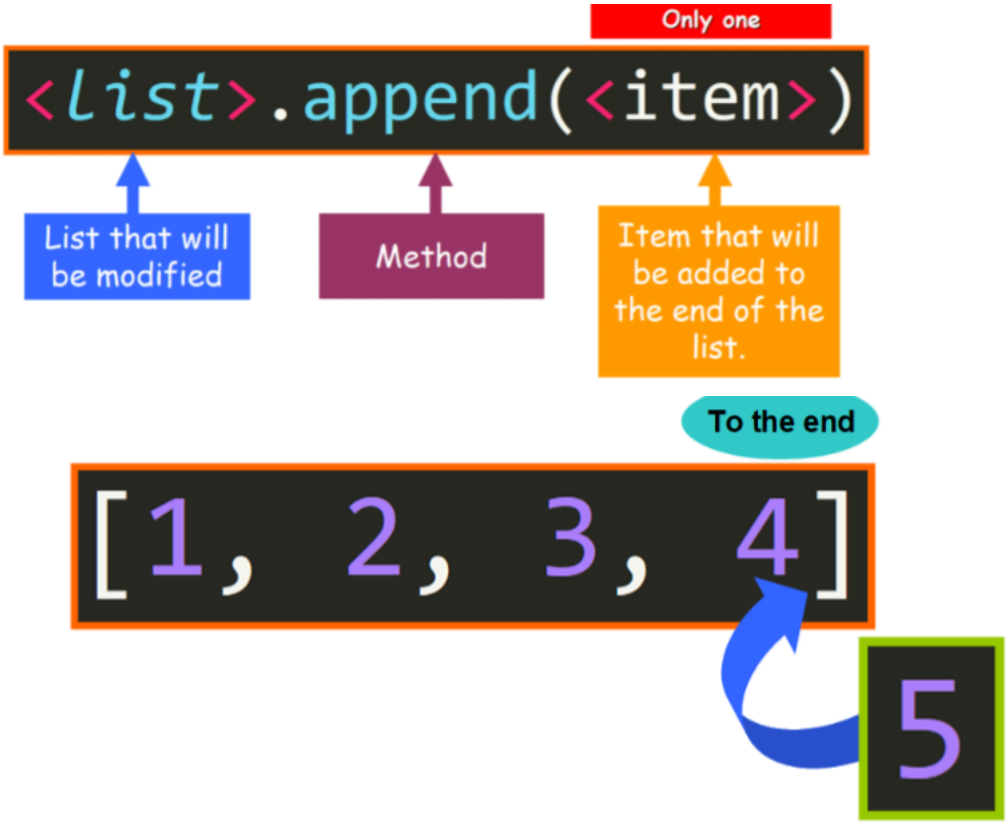
append()

- `append()` is a Python list method
- Used to add a single element at the end of a list
- Modifies the original list
- It is an in-place operation
- Adds the complete object as one element
- Accepts any datatype
- Syntax:

```python
list.append(value)
```

- Time Complexity: `O(1)` average case
- Faster than `insert()`

  - modifies original list
  - adds only ONE element at a time
  - returns `None`



In [4]:
```
l = []# empty
print(l)

l.append(10)

print(l)
```

```
[]
[10]
```

In [73]:
```
lst = [3,4,5]
# append function to add an element at the end of the list.
lst.append(6)
print (lst)
```

```
[3, 4, 5, 6]
```

In [8]:
```
original_lst = [3,4,5]
#.
original_lst.append(6,7)
print (original_lst)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[8], line 4
      2 original_lst = [3,4,5]
      3 #use the append function to add an element at the end of the list.
----> 4 original_lst.append(6,7)
      5 print (original_lst)

TypeError: list.append() takes exactly one argument (2 given)
```

In [14]:
```
original_lst = [3,4,5]

#tuple fnction.
original_lst.append((6,7))
print (original_lst)
```

```
[3, 4, 5, (6, 7)]
```

In [75]:
```
original_lst = [4, 5, 6]

original_lst.append([7, 8])
print (original_lst)
```

```
[4, 5, 6, [7, 8]]
```

In [77]:
```
names = ["shiv", "ram", "krisna", "ganesh"]
#tuple.
names.append(("prakash", "naresh"))

print(names)
```

```
['shiv', 'ram', 'krisna', 'ganesh', ('prakash', 'naresh')]
```

In [79]:
```
names = ["shiv", "ram", "krisna", "ganesh"]
#use the append function  list.
names.append(["prakash", "naresh"])
print(original_names)
```

```
['shiv', 'ram', 'krisna', 'ganesh', ['prakash', 'naresh']]
```

In [64]:
```
# append
L = [1,2,3,4,5]
L.append(True)
print(L)
```

```
[1, 2, 3, 4, 5, True]
```

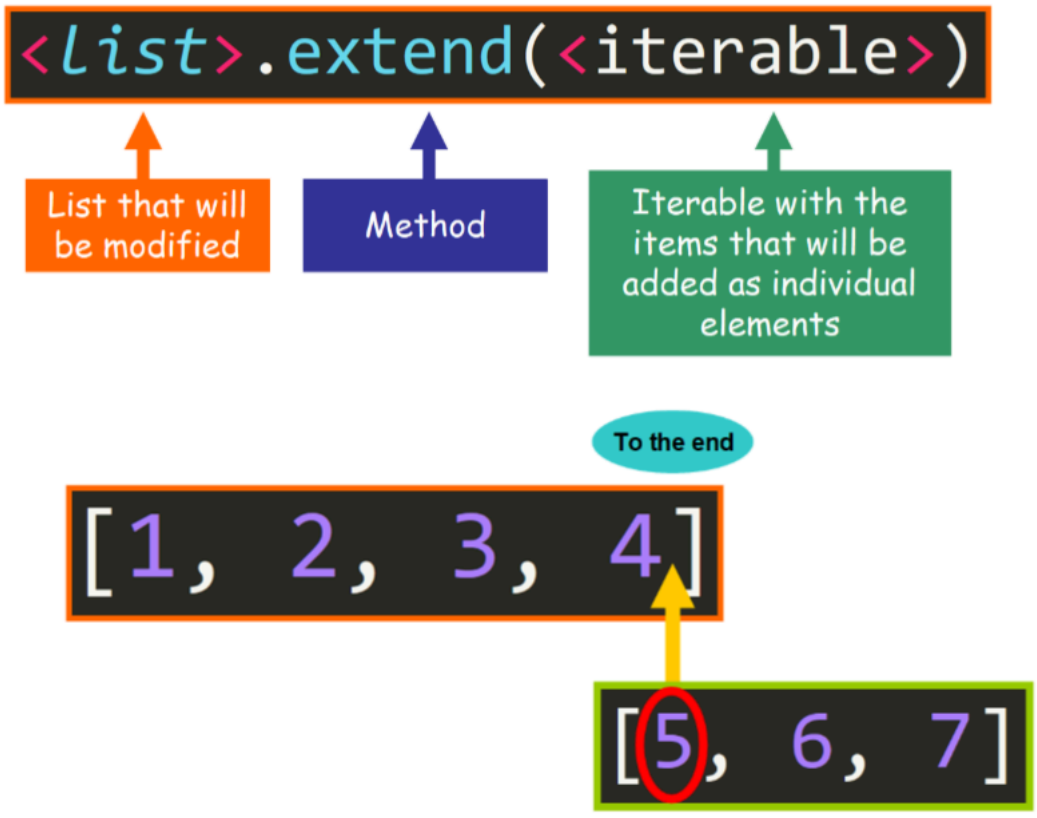
==========

extend()

- `extend()` is a Python list method
- Used to add elements of an iterable to a list
- Modifies the original list
- It is an in-place operation
- Adds elements one by one
- Accepts any iterable (`list` , `tuple` , `set` , `string` , etc.)
- More memory efficient than `+` operator
- Syntax:

Python
```
list.extend(iterable)
```

- Time Complexity: `O(n)`



`extend()` is used to add multiple elements to a list.

It adds elements one by one at the end of the list.

👉 extend means Expand the list with multiple values.

In [38]:
```
l = [1, 2]

l.extend([3, 4])

print(l)
```

```
[1, 2, 3, 4]
```

In [40]:
```
l = [1,2]

l.extend("ABC")

print(l)
```

```
[1, 2, 'A', 'B', 'C']
```

In [44]:
```
## Combining multiple lists
java = ["Spring", "Hibernate"]
python = ["NumPy", "Pandas"]

python.extend(java)

print(python)
```

```
['NumPy', 'Pandas', 'Spring', 'Hibernate']
```

In [49]:
```
a = [1, 2, 3, 4]
b = [5, 6, 7]

a.extend(b)
print (a)
print(b)
```

```
[1, 2, 3, 4, 5, 6, 7]
[5, 6, 7]
```

In [119…
```
a = [1,2,3]

a.extend(a[1:])

print(a)
```

```
[1, 2, 3, 2, 3]
```

extend() only change target list (a). Source list unchanged (b) .

In [55]:
```
a = [1, 2, 3, 4]# list
b = (1, 2, 3, 4)# tuple

a.extend(b)
print(a)
print(b)
```

```
[1, 2, 3, 4, 1, 2, 3, 4]
(1, 2, 3, 4)
```

In [59]:
```
a = [1, 2, 3, 4]# list
b = (1, 2, 3, 4)# tuple
c={6,7,9} # set

a.extend(b)
a.extend(c)
print(a)
print(b)
print(c)
```

```
[1, 2, 3, 4, 1, 2, 3, 4, 9, 6, 7]
(1, 2, 3, 4)
{9, 6, 7}
```

In [69]:
```
# List that will be extended
a = ["a", "b", "c"]

# String that will be used to extend the list
```

```
b = "Hello, World!"

# Method call
a.extend(b)

# The value of a was updated
print(a)
```

```
['a', 'b', 'c', 'H', 'e', 'l', 'l', 'o', ',', ' ', 'W', 'o', 'r', 'l', 'd', '!']
```

In [71]:
```
# List that will be extended
a = ["a", "b", "c"]

# Dictionary that will be us
b = {"d": 5, "e": 6, "f": 7}

# Method call
a.extend(b)

print(a)
```

```
['a', 'b', 'c', 'd', 'e', 'f']
```

---

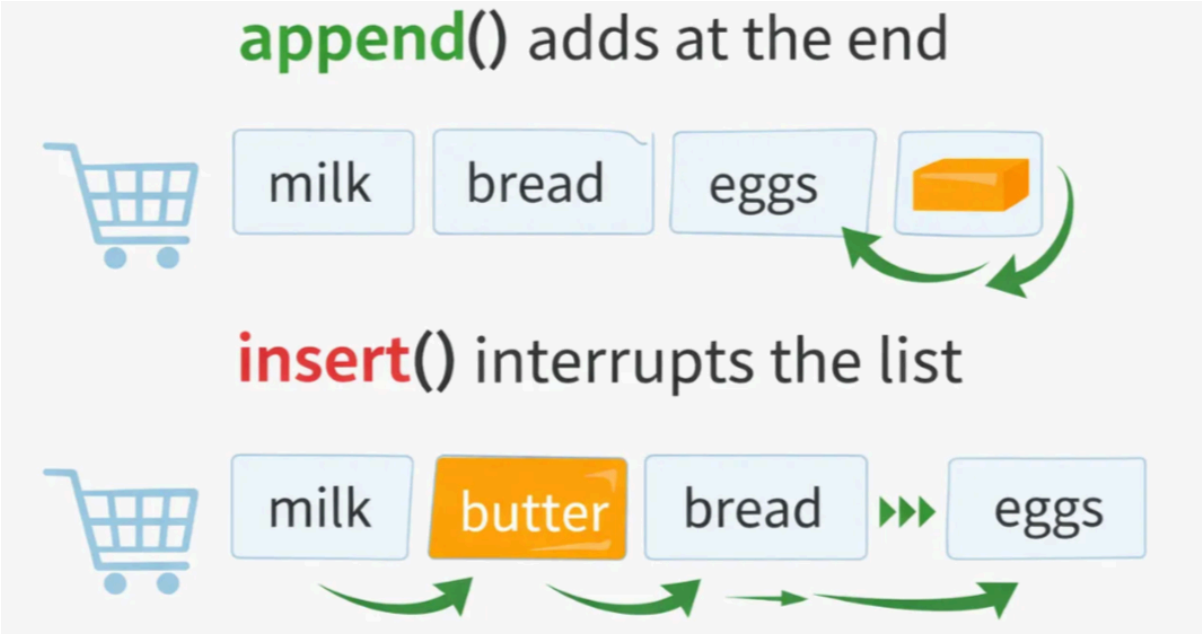
insert()

- `insert()` is a Python list method
- Used to add an element at a specific index
- It modifies the original list
- It is an in-place operation
- Existing elements shift to the right
- Syntax:

Python

```
list.insert(index, value)
```

- Accepts any datatype
- Supports negative indexing
- Time Complexity: `O(n)` because shifting may occur
- Slower than `append()`



In [85]:
```
fruits = ['apple', 'banana', 'mango']

# Insert 'cherry' at index 1
fruits.insert(1, 'cherry')

print(fruits)
```

```
['apple', 'cherry', 'banana', 'mango']
```

In [89]:
```
numbers = [1, 3, 4]
numbers.insert(1, 2)  # Inserts '2' at index 1
print(numbers)
```

```
[1, 2, 3, 4]
```

In [107…
```
a = [1, 2, 3,4]

a.insert(len(a), 4)

print(a)
```

```
[1, 2, 3, 4, 4]
```

In [109…
```
a = [1, 2, 3]

a.insert(-1, 100)

print(a)
```

```
[1, 2, 100, 3]
```

In [111…
```
india = ["Rohit Sharma", "Virat Kohli"]

india.insert(1, ("MS Dhoni", "Sachin Tendulkar"))

print(india)
```

```
['Rohit Sharma', ('MS Dhoni', 'Sachin Tendulkar'), 'Virat Kohli']
```

In [ ]:
```

```

In [91]:
```
india = []

india.insert(0, 'Rohit sharma')

print(india)
```

```
['Rohit sharma']
```

In [97]:
```
india = []

india.insert(0, 'Rohit sharma')
india.insert(0,"virat kohli")
print(india)
```

```
['virat kohli', 'Rohit sharma']
```

In [99]:
```
india.append("jaspreet")
india.insert(0,"shreysh")
india.extend(("abhishek","sanju","dube"))
print(india)
```

```
['shreysh', 'virat kohli', 'Rohit sharma', 'jaspreet', 'abhishek', 'sanju', 'dube']
```

In [101…
```
india = [[1, 2], [4, 5]]
india.insert(1, [3, 4])  # Inserts a new row at index 1
```

```
print(india)
```

```
[[1, 2], [3, 4], [4, 5]]
```

In [105…
```
india = [[1, 2], ["sachin", "sehwag"]]
india.insert(1, ["saurav", 4+3j])  # Inserts a new row at index 1
print(india)
```

```
[[1, 2], ['saurav', (4+3j)], ['sachin', 'sehwag']]
```

In [113…
```
a = [1, 2, 3, 4, 5]
b = a[1:4]

a.insert(0, b)

print(a)
```

```
[[2, 3, 4], 1, 2, 3, 4, 5]
```

In [115…
```
india = [
    "Rohit",
    "Virat",
    "Dhoni",
    "Hardik",
    "Jadeja"
]

top_players = india[0:3]

india.insert(2, top_players)

print(india)
```

```
['Rohit', 'Virat', ['Rohit', 'Virat', 'Dhoni'], 'Dhoni', 'Hardik', 'Jadeja']
```

In [117…
```
a = [1, 2, 3, 4]

rev = a[::-1]

a.insert(1, rev)

print(a)
```

```
[1, [4, 3, 2, 1], 2, 3, 4]
```

Features of `append()`

- ✅ Original list बदलते
- ✅ In-place operation
- ✅ Memory efficient
- ❌ Iterable unpack करत नाही
- ✅ Single object add करतो
- ✅ Any datatype accept करतो

Features of `extend()`

- ✅ Original list बदलते
- ✅ In-place operation
- ✅ Memory efficient
- ✅ Any iterable accept करत

Features of `insert()` in Python

- ✅ Adds element at a specific index
- ✅ Modifies the original list
- ✅ In-place operation
- ✅ Shifts existing elements to the right
- ✅ Accepts any datatype
- ✅ Supports negative indexing
- ✅ Can insert at beginning, middle, or end
- ✅ Can insert complex objects (list, tuple, dictionary, etc.)
- ❌ Slower than `append()` because shifting is required
- ✅ Returns `None`

Easy Trick 😄

- `append()` → "box add करतो"
- `extend()` → "box उघडून items add करतो"

insert()	append()	extend()
Used to add the element at the desired position in the list	Used to add the element at the end of the list	Used to add the elements from the iterable at the end of the list
`list.insert(index, element)`	`list.append(element)`	`list.extend(iterable)`
Takes two parameters	Takes a single parameter	Takes a single iterable parameter
Can take iterable but adds it as it is	Can take iterable but adds it as it is	Takes iterable and each item is added individually
List length increases by 1	List length increases by 1	Length of the list increases by the number of items in the iterable
Time complexity is constant i.e., O(1)	Time complexity is linear i.e., O(n)	Has time complexity of O(x), where **x** is the length of the iterable

---

remove()

- `remove()` is a built-in list method in Python.
- It removes the **first occurrence** of a specified element.
- It modifies the **original list** directly (in-place operation).
- It removes elements by **value**, not by index.
- It takes only **one argument** → the element to remove.
- Syntax:

```python
list.remove(element)
```

- If multiple same elements exist, only the **first matching element** is removed.
- If the element is not found, Python raises:

```python
ValueError
```

- `remove()` returns `None`.
- Time Complexity: `O(n)` because searching is linear.

In [17]:
```
a = [10, 20, 30, 20]

a.remove(20)

print(a)
```

```
[10, 30, 20]
```

In [25]:
```
india = ["Rohit", "Virat", "Gill"]

india.remove("Virat")

print(india)
```

```
['Rohit', 'Gill']
```

In [27]:
```
 # Passing list as argument in remove()
a = [1, 2, 3, 4, 5, 6, 7, 8]
b = [2, 4, 6]

for item in b:
    if item in a:
        a.remove(item)

print(a)
```

```
[1, 3, 5, 7, 8]
```

In [29]:
```
# Trying to delete Element that doesn't Exist
a = [ 'a', 'b', 'c', 'd' ]

a.remove('e')
print(a)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[29], line 4
      1 # Trying to delete Element that doesn't Exist
      2 a = [ 'a', 'b', 'c', 'd' ]
----> 4 a.remove('e')
      5 print(a)

ValueError: list.remove(x): x not in list
```

---

clear()

`clear()` is a Python list method used to remove **all elements** from a list.

After using `clear()`, the list becomes empty.

Syntax

Python

```
list.clear()
```

- Removes **all items** from the list
- Makes the list empty
- Modifies the original list directly
- Performs in-place operation
- Takes **no arguments**
- Returns `None`
- Works only with mutable lists

In [2]:
```
a = [10, 20, 30, 40]

a.clear()

print(a)
```

```
[]
```

In [6]:
```
# Memory address remain same.
a = [1, 2, 3]

print(id(a))   # Before clear

a.clear()

print(a)
print(id(a))   # After clear
```

```
3152213040896
[]
3152213040896
```

---

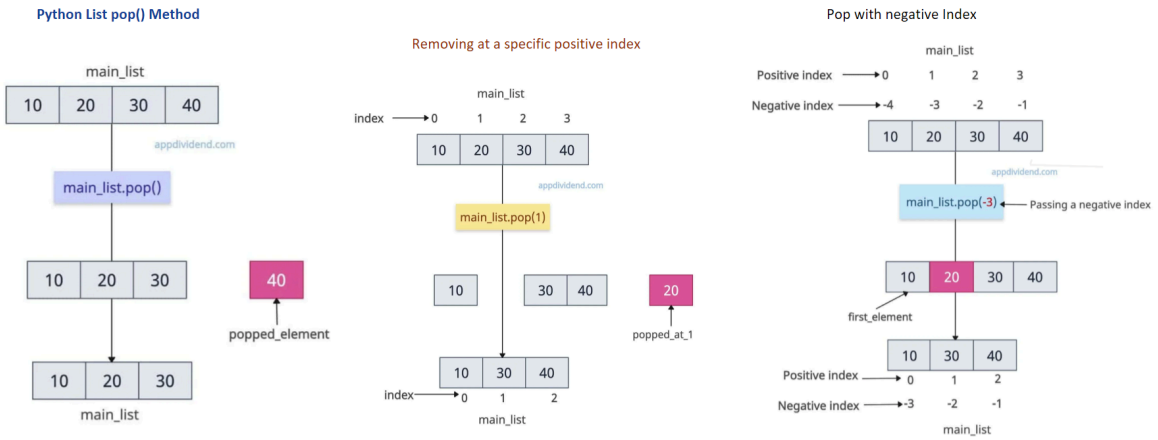
pop()

- `pop()` removes an element using index
- Removed value is returned
- Default → removes last element (`-1`)
- Original list is modified (in-place)
- `IndexError` occurs on empty list
- `IndexError` occurs for invalid index
- Widely used in stack implementation
- `pop()` on last element → fast (`O(1)`)
- `pop(0)` → slow because of shifting (`O(n)`)

Python

```
list.pop(index)
```

- `index` is optional
- Default → removes the last element



In [11]:
```
main_list = [10, 20, 30, 40]

popped_element = main_list.pop()

print(popped_element)
print(main_list)
```

```
40
[10, 20, 30]
```

Explanation

- `pop()` removes the last element by default
- Removed element is stored in `popped_element`
- Original list gets modified automatically

In [14]:
```
main_list = [10, 20, 30, 40]

popped_at_1 = main_list.pop(1)
# Removes and returns 20
```

```
print(popped_at_1)

# Output: 20

print(main_list)

# Output: [10, 30, 40]
```

```
20
[10, 30, 40]
```

In [20]:
```
main_list = [10, 20, 30, 40]

first_element = main_list.pop(0)
# Removes and returns 10

print(first_element)

print(main_list)
```

```
10
[20, 30, 40]
```

In [22]:
```
main_list = [10, 20, 30, 40] # negative index

backward_element = main_list.pop(-3)

print(backward_element)



print(main_list)
```

```
20
[10, 30, 40]
```

---

del

del is a Python keyword used to delete list elements, slices, or the entire list.

- `del` is a keyword
- Removes element using index
- Can delete slices and whole variable
- Does not return value
- Original list modifies directly

In [31]:
```
a = [1, 2, 3]

del a

print(a)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[31], line 5
      1 a = [1, 2, 3]
      3 del a
----> 5 print(a)

NameError: name 'a' is not defined
```

In [27]:
```
a = [10, 20, 30, 40]

del a[1]

print(a)
```

```
[10, 30, 40]
```

In [29]:
```
a = [10, 20, 30, 40, 50]

del a[1:4]

print(a)
```

```
[10, 50]
```

In [33]:
```
a = [10, 20, 30, 40]

del a[-1]
```

```
print(a)
```

```
[10, 20, 30]
```

In [35]:
```
a = [10, 20, 30, 40, 50]

del a[-3:-1]

print(a)
```

```
[10, 20, 50]
```

==========

In [ ]:

In [24]:
```
main_list = [1, 2, 3, 2]

removed = main_list.remove(2)
# Removes first occurrence of 2

print(removed)
# Output: None

print(main_list)
# Output: [1, 3, 2]


del main_list[0]
# Removes element at index 0

print(main_list)
# Output: [3, 2]


popped = main_list.pop(1)
# Removes and returns element at index 1

print(main_list)
# Output: [3]

print(popped)
# Output: 2
```

```
None
[1, 3, 2]
[3, 2]
[3]
2
```

Difference Between `remove()`, `pop()`, `clear()`, and `del`

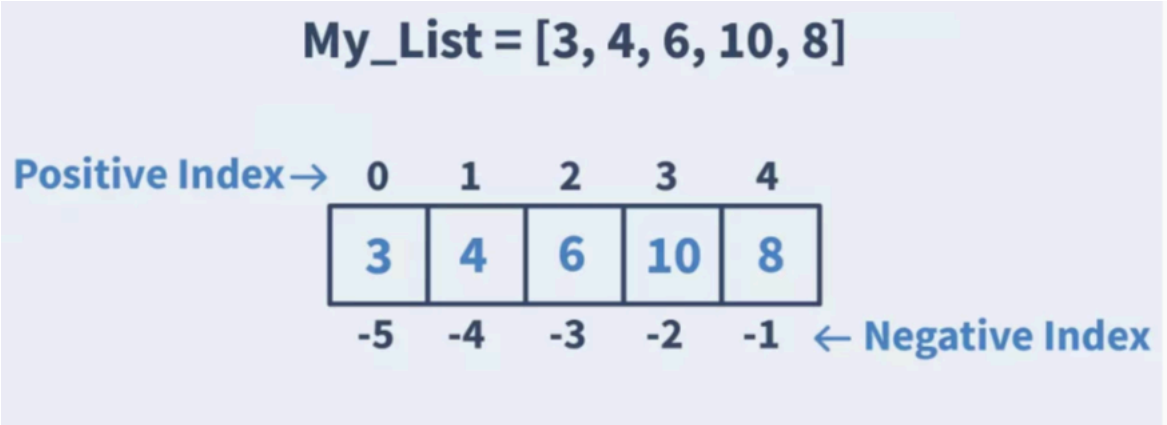
Feature	`remove()`	`pop()`	`clear()`	`del`
Type	Method	Method	Method	Keyword
Removes by	Value	Index	All elements	Index / slice / variable
Return value	`None`	Removed element	`None`	No return
Default behavior	No default	Removes last element	Clears list	No default
Modifies original list	✅ Yes	✅ Yes	✅ Yes	✅ Yes
Can delete whole list	❌ No	❌ No	❌ No	✅ Yes
Supports slicing	❌ No	❌ No	❌ No	✅ Yes
Error condition	Value not found → `ValueError`	Invalid index → `IndexError`	No error on empty list	Invalid index → `IndexError`

==========

index

An **index** is the position number of an element in a list.

- Index starts from `0`
- Negative index starts from `-1`



In [60]:
```
# Creating List
a = [10, 20, 30, 40, 50]

print("Original List:", a)


# Positive Indexing
print("\nPositive Indexing")
print(a[0])   # First element
print(a[1])   # Second element
print(a[4])   # Last element


# Negative Indexing
print("\nNegative Indexing")
print(a[-1])  # Last element
print(a[-2])  # Second last
print(a[-5])  # First element


# Modify Using Index
print("\nModify Using Index")
a[1] = 99
print(a)


# Slicing
print("\nSlicing")
print(a[1:4])    # Index 1 to 3
print(a[:3])     # Start to index 2
print(a[2:])     # Index 2 to end
print(a[::-1])   # Reverse list


# Delete Using Index
print("\nDelete Using Index")
del a[1]
print(a)


# Delete Using Negative Index
print("\nDelete Using Negative Index")
del a[-1]
print(a)


# Pop Using Index
print("\nPop Using Index")
x = a.pop(1)
print("Removed:", x)
print(a)


# Index Error Example
print("\nIndex Error Example")
# print(a[10])   # Uncomment to see IndexError
```

```
Original List: [10, 20, 30, 40, 50]

Positive Indexing
10
20
50

Negative Indexing
50
40
10

Modify Using Index
[10, 99, 30, 40, 50]

Slicing
[99, 30, 40]
[10, 99, 30]
[30, 40, 50]
[50, 40, 30, 99, 10]

Delete Using Index
[10, 30, 40, 50]

Delete Using Negative Index
[10, 30, 40]

Pop Using Index
Removed: 30
[10, 40]

Index Error Example
```

In [62]:
```python
animals = ['cat', 'dog', 'rabbit', 'horse']

# get the index of 'dog'
index = animals.index('dog')

print(index)
```

```
1
```

In [58]:
```python
a = "Python"

print("Positive Indexing")
print(a[0])
print(a[1])

print("\nNegative Indexing")
print(a[-1])
print(a[-2])
```

```
Positive Indexing
P
y

Negative Indexing
n
o
```

In [ ]:
```
```

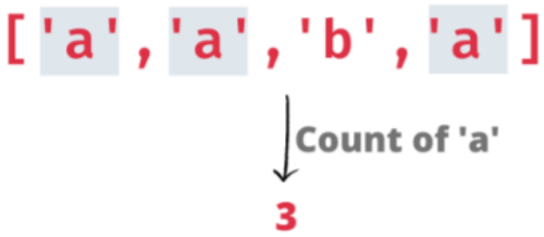
---

count

- `count()` is a list method
- Counts occurrence of a specific value
- Returns integer value
- Does not modify original list
- Returns `0` if value not found
- Case-sensitive for strings
- Works with numbers, strings, lists, etc.

Python

```
list.count(value)
```



In [65]:
```
a = [1, 2, 1, 3, 1]

print(a.count(1))
```

```
3
```

In [67]:
```
a = [1, 'GfG', 3.14, 'GfG', 1, True]

c1 = a.count('GfG')
c2 = a.count(1)

print(c1)
print(c2)
```

```
2
3
```

In [69]:
```
s = "python is easy to learn and python is powerful".split()

c1 = s.count("python")
c2 = s.count("is")

print(c1)
print(c2)
```

```
2
2
```

---

sort

reverse

---

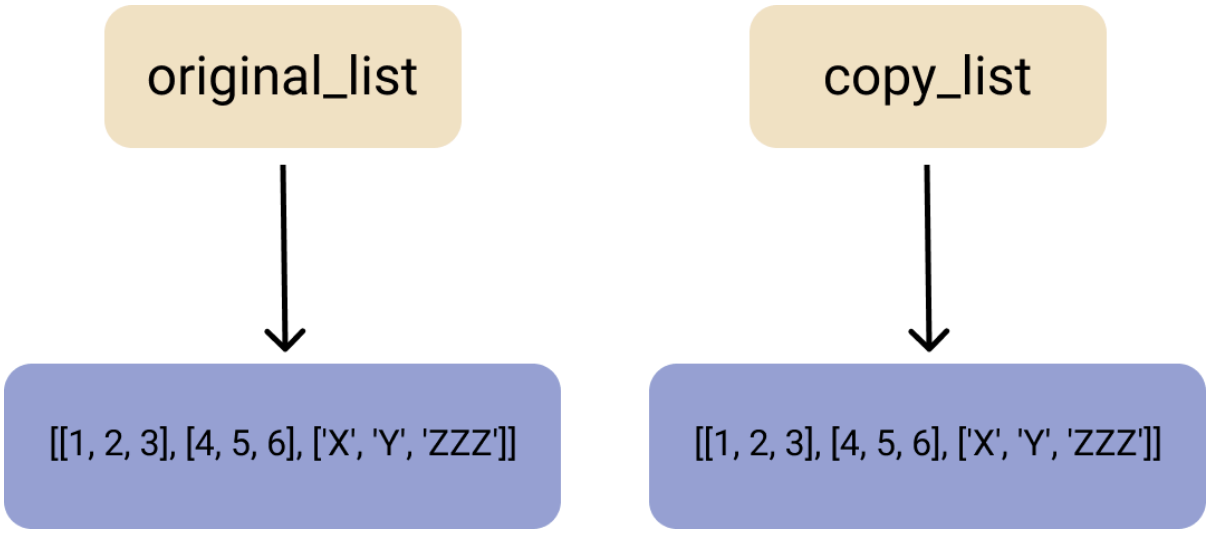
copy

- `copy()` creates a **shallow copy** of a list.
- It returns a **new list object**.
- Original and copied lists have **different memory addresses**.
- Changes in outer elements do not affect the original list.
- Nested mutable objects are still shared in shallow copy.
- `copy()` works only for list objects.
- Syntax:

```python
new_list = old_list.copy()
```

- `=` does not create a copy; it creates a reference.
- `copy()` is better than assignment when independent lists are needed.
- Deep copy can be created using:

```python
import copy
copy.deepcopy()
```



In [29]:
```python
a = [1, 2, 3]

print(a)
print(id(a))
```

```
[1, 2, 3]
1962315539520
```

In [33]:
```python
# Reference Copy (=)
a = [1, 2, 3]

b = a

print(a)
print(b)

print(id(a))
print(id(b))
```

```
[1, 2, 3]
[1, 2, 3]
1962315534080
1962315534080
```

In [35]:
```python
# copy() Method
a = [1, 2, 3]

b = a.copy()
```

```
print(a)
print(b)

print(id(a))
print(id(b))
```

```
[1, 2, 3]
[1, 2, 3]
1962315480832
1962315469184
```

In [37]:
```
# List Slicing Copy ([:])
a = [1, 2, 3]

b = a[:]

print(a)
print(b)

print(id(a))
print(id(b))
```

```
[1, 2, 3]
[1, 2, 3]
1962315401344
1962315402240
```

In [ ]:

In [7]:
```
    mylist = ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']
```

In [11]:
```
mylist1 = mylist # # Create a new reference "mylist1"
```

In [13]:
```
id(mylist) , id(mylist1) # The address of both mylist & mylist1 will be the same
```

Out[13]:
```
(1962315474880, 1962315474880)
```

In [15]:
```
mylist2 = mylist.copy() # Create a copy of the list
```

In [17]:
```
id(mylist2) # The address of mylist2 will be different from mylist because mylist
```

Out[17]:
```
1962315531648
```

In [19]:
```
mylist[0] = 1
```

In [21]:
```
mylist
```

Out[21]:
```
[1, 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']
```

In [23]:
```
mylist1 # mylist1 will be also impacted as it is pointing to the same list
```

Out[23]:
```
[1, 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']
```

In [27]:
```
mylist2 # Copy of list won't be impacted due to changes made on the original list
```

Out[27]:
```
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']
```

==========

In [ ]:

In [ ]:
```
interview main point
```

In [ ]:

Real Project Usage Questions

Why use list instead of tuple? When should you avoid list? How to optimize large lists? Difference between NumPy array and list? Why queue using deque instead of list? How lists behave in multithreading?

WAP to enter 5 numbers from user and add them into a list and print the list

In [69]:
```
i = 0
l = []

while i < 5:
    n = int(input("Enter a number: "))
    l.append(n)
    i += 1

print(l)
```

```
[10, 30, 49, 50, 44]
```

WAP to enter 6numbers from user, append them into a list and print sum of them, whithout using sum function

In [74]:
```
i = 0
l = []
```

```
s = 0

while i < 6:
    n = int(input("Enter a number: "))
    l.append(n)
    s = s + l[i]
    i += 1

print("Sum = ", s)
```

```
Sum =  287
```

python program to print odd and even numbers from the list of integers

In [77]:
```
numbers = [22,11,34,56,78,61,35,91,77,59]

odd_list = list()
even_list = list()
odd_count = 0
even_count = 0

for i in numbers:
    if i % 2 == 0:
        even_list.append(i)
        even_count += 1
    else:
        odd_list.append(i)
        odd_count += 1

print("There are {} odd and {} even elements".format(odd_count,even_count))
print("Odd elements: ", odd_list)
print("Even elements: ", even_list)
```

```
There are 6 odd and 4 even elements
Odd elements:  [11, 61, 35, 91, 77, 59]
Even elements:  [22, 34, 56, 78]
```

In [ ]:
```

```